

Tổng quan chương

Chương 1 đã phác thảo tổng quan định hướng giải pháp của đề tài tương tác 3D. Để xây dựng hệ thống hoạt động chính xác và ổn định, chương này sẽ tập trung trình bày các cơ sở lý thuyết và thuật toán nền tảng. Các nội dung bao gồm: (i) tiền xử lý ảnh với thư viện OpenCV, (ii) kỹ thuật trích xuất 21 điểm mốc bàn tay bằng thư viện MediaPipe, (iii) giải thuật chuẩn hóa tọa độ và mô hình mạng nơ-ron nhận dạng cử chỉ tĩnh/động, (iv) nguyên lý truyền dữ liệu thời gian thực qua socket UDP, và (v) cơ sở toán học của bộ lọc thích nghi chống rung nhiễu One Euro Filter.

0.1 Tiền xử lý ảnh với OpenCV

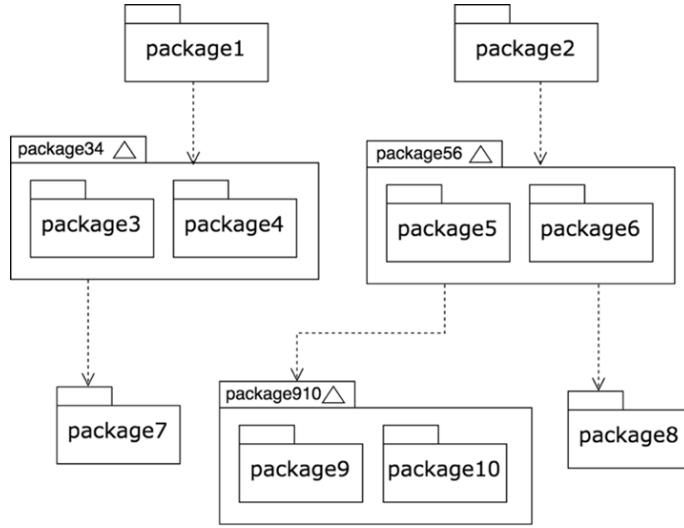
Trong hệ thống tương tác cử chỉ tay trực tuyến, việc thu nhận và tiền xử lý luồng video đầu vào từ webcam đóng vai trò khởi đầu quan trọng. Thư viện OpenCV **opencv_library** được sử dụng để tương tác với phần cứng camera, chụp các khung hình (frames) liên tiếp ở độ phân giải xác định (ví dụ: 960×540 pixel) và chuyển đổi định dạng dữ liệu ảnh.

Do hình ảnh từ webcam thường bị ngược gương so với chuyển động thực tế của người dùng, khung hình thu được trước hết phải được lật theo chiều ngang (flip) bằng thuật toán biến đổi hình học trong OpenCV để đảm bảo tính trực quan khi tương tác. Tiếp theo, OpenCV hỗ trợ chuyển đổi không gian màu từ mặc định của hệ thống (BGR) sang không gian màu RGB. Đây là định dạng chuẩn yêu cầu bởi các mô hình học sâu của MediaPipe nhằm tiến hành dò tìm bàn tay ở các bước tiếp theo.

0.2 Trích xuất đặc trưng khớp tay bằng MediaPipe Hands

Để hệ thống có thể hiểu được cấu trúc bàn tay mà không cần các cảm biến chiều sâu đắt tiền, công nghệ trích xuất đặc trưng bàn tay dựa trên mô hình học sâu của Google MediaPipe **lugaresi2019mediapipe** được áp dụng. Giải pháp MediaPipe Hands sử dụng một đường ống xử lý (pipeline) gồm hai mô hình học sâu hoạt động liên tiếp:

1. ****Mô hình phát hiện bàn tay (Hand Detector):**** Chạy trên toàn bộ khung hình để xác định vùng bao (bounding box) của bàn tay. Việc này giúp giảm không gian tính toán cho các frame tiếp theo bằng cách chỉ định vị vùng chứa tay thay vì quét toàn bộ ảnh.
2. ****Mô hình dự đoán điểm mốc khớp tay (Hand Landmark Model):**** Chạy trên vùng ảnh bàn tay đã được cắt ra từ mô hình phát hiện để dự đoán tọa độ không gian 3D tương đối của 21 điểm khớp mốc (landmarks), được đánh số từ 0 đến 20 như mô tả trong Hình 0.1.



Hình 0.1: Sơ đồ đánh số 21 điểm landmark bàn tay trong MediaPipe

Dữ liệu đầu ra của MediaPipe đối với mỗi điểm landmark i ($i \in [0, 20]$) bao gồm ba tọa độ (x_i, y_i, z_i) , trong đó x_i và y_i đại diện cho vị trí pixel trên ảnh (đã được chuẩn hóa về khoảng $[0.0, 1.0]$ theo chiều rộng và chiều cao khung hình), còn z_i biểu thị độ sâu tương đối của khớp so với khớp cổ tay (Wrist - điểm số 0).

0.3 Nhận dạng cử chỉ tay

Hệ thống kết hợp cả phương pháp heuristics dựa trên luật hình học (cho các tương tác cơ bản, phản hồi nhanh) và phương pháp học máy có huấn luyện (cho các cử chỉ tĩnh phức tạp và cử chỉ động theo thời gian).

0.3.1 Phương pháp không huấn luyện (Rule-based / Heuristics)

Phương pháp này dựa trên trạng thái co/duỗi của các ngón tay để đưa ra quyết định nhanh về cử chỉ. Trạng thái của bốn ngón tay dài (trỏ, giữa, áp út, út) được xác định bằng cách so sánh khoảng cách Euclide từ điểm cổ tay (landmark 0) đến điểm đầu ngón tay (tip) và điểm khớp giữa (PIP). Đối với một ngón tay f có đầu ngón là Tip_f và khớp giữa là Pip_f : - Nếu $\|P_{Tip_f} - P_0\| > \|P_{Pip_f} - P_0\|$, ngón tay được coi là đang ****duỗi**** (đánh dấu giá trị bằng 1). - Ngược lại, ngón tay được coi là đang ****co**** (đánh dấu giá trị bằng 0).

Đối với ngón tay cái, do chuyển động đặc thù theo chiều ngang, hệ thống tính khoảng cách giữa điểm đầu ngón cái (landmark 4) với gốc ngón út (landmark 17). Nếu khoảng cách này vượt quá một ngưỡng tỉ lệ so với chiều rộng bàn tay (khoảng cách landmark 5 đến 17), ngón cái được coi là duỗi. Tập hợp trạng thái co/duỗi của 5 ngón tạo thành một mảng nhị phân giúp ánh xạ nhanh ra các cử chỉ cơ bản như Open Hand $([1, 1, 1, 1, 1])$, Close/Fist $([0, 0, 0, 0, 0])$, hay Pointer $([0, 1, 0, 0, 0])$.

0.3.2 Thuật toán chuẩn hóa tọa độ landmark

Khi sử dụng mô hình học máy, tọa độ thô từ MediaPipe phụ thuộc lớn vào vị trí của bàn tay trong khung hình (trái/phải, trên/dưới) và khoảng cách từ tay đến camera (tay ở gần sẽ có khoảng cách giữa các khớp lớn hơn tay ở xa). Để đảm bảo mô hình phân loại bất biến với các yếu tố này, thuật toán chuẩn hóa tọa độ được thực hiện theo các bước sau:

Đầu tiên, tịnh tiến toàn bộ 21 điểm landmark $P_i = (x_i, y_i, z_i)$ sao cho điểm cổ tay P_0 trùng với gốc tọa độ $(0, 0, 0)$:

$$P'_i = P_i - P_0 = (x_i - x_0, y_i - y_0, z_i - z_0), \quad \forall i \in [0, 20] \quad (1)$$

Sau đó, chuyển đổi mảng tọa độ 3D thành mảng một chiều kích thước 42 phần tử chứa các cặp tọa độ (x'_i, y'_i) (đối với mô hình tĩnh chỉ cần tọa độ 2D phẳng). Tìm giá trị tuyệt đối lớn nhất trong mảng một chiều này để làm nhân tố tỉ lệ d_{max} :

$$d_{max} = \max_j |V_j| \quad (2)$$

Trong đó V là mảng một chiều chứa $21 \times 2 = 42$ tọa độ tịnh tiến.

Cuối cùng, chia toàn bộ các phần tử trong mảng cho d_{max} để thu được mảng đặc trưng chuẩn hóa V'' :

$$V''_j = \frac{V_j}{d_{max}}, \quad \forall j \in [0, 41] \quad (3)$$

Nhờ thuật toán chuẩn hóa này, các giá trị đặc trưng đầu vào của mô hình luôn nằm trong khoảng $[-1.0, 1.0]$ bất kể bàn tay đang ở vị trí nào hay to nhỏ ra sao.

0.3.3 Phân loại cử chỉ tĩnh bằng mạng nơ-ron truyền thẳng (MLP)

Với cử chỉ tĩnh (như Open, Close, Pointer, OK, Peace, ThumbsUp), hệ thống sử dụng một mạng nơ-ron nhân tạo dạng truyền thẳng nhiều lớp MLP (Multi-Layer Perceptron) với kiến trúc gồm: - **Lớp đầu vào:** 42 nơ-ron nhận dữ liệu đặc trưng đã chuẩn hóa V'' . - **Các lớp ẩn:** Gồm các lớp Fully Connected kết hợp hàm kích hoạt ReLU và Dropout để chống quá khớp (overfitting). - **Lớp đầu ra:** Lớp Softmax gồm 6 nơ-ron tương ứng với xác suất của các phân lớp cử chỉ. Mô hình sau khi huấn luyện trên Python được chuyển đổi sang định dạng TensorFlow Lite ('.tflite') **tflite2017** để tối ưu hóa tốc độ suy luận thời gian thực trên CPU máy tính cá nhân.

0.3.4 Phân loại cử chỉ động

Nhận dạng cử chỉ động (như Swipe Left, Swipe Right, Circle,...) yêu cầu xử lý thông tin chuỗi theo thời gian. Hệ thống lưu trữ lịch sử tọa độ của điểm đầu ngón trỏ (landmark 8) trong một hàng đợi có kích thước cố định $N = 32$ khung hình gần nhất. Chuỗi tọa độ này được chuẩn hóa tương đối so với điểm bắt đầu của chuỗi và kích thước khung hình camera:

$$x_k^{norm} = \frac{x_k - x_{start}}{W}, \quad y_k^{norm} = \frac{y_k - y_{start}}{H}, \quad \forall k \in [0, N - 1] \quad (4)$$

Trong đó W, H lần lượt là chiều rộng và chiều cao của khung hình camera. Mảng đặc trưng một chiều gồm $32 \times 2 = 64$ phần tử này được đưa vào mô hình học máy thứ hai để phân loại cử chỉ động đang diễn ra.

0.4 Giao thức truyền dữ liệu UDP

Để tích hợp module AI viết bằng Python với môi trường tương tác 3D Unity viết bằng C#, một cơ chế truyền thông điệp liên tiến trình độ trễ thấp là bắt buộc. Hệ thống sử dụng socket mạng qua giao thức UDP (User Datagram Protocol) **rfc768udp**.

Khác với TCP đòi hỏi bắt tay kết nối và đảm bảo truyền tuần tự không mất mát (gây trễ tích lũy khi có gói tin bị mất), UDP là giao thức phi kết nối (connection-less), truyền các gói tin độc lập với tốc độ tối đa của luồng dữ liệu. Trong hệ thống tương tác cử chỉ trực tuyến, dữ liệu liên tục được cập nhật ở tần số cao (30 – 60 FPS). Nếu một gói tin chứa tọa độ bị mất, gói tin tiếp theo sẽ ngay lập tức bù đắp mà không cần truyền lại, giúp hệ thống duy trì tính thời gian thực và loại bỏ hoàn toàn độ trễ tích lũy.

0.5 Bộ lọc thích nghi chống rung nhiễu One Euro Filter

Tọa độ thu được từ MediaPipe thường chứa nhiễu tần số cao do giới hạn độ phân giải của camera, rung động vật lý từ tay người dùng, hoặc sai số từ mô hình học sâu. Hiện tượng này gây ra sự rung lắc dữ liệu (jitter) ở phía Unity, làm cho mô hình bàn tay ảo hoặc lưỡi dao chém hoa quả bị giật.

Để loại bỏ hiện tượng trên mà không tạo ra độ trễ lớn khi di chuyển nhanh, bộ lọc thích nghi One Euro Filter **casiez2012one** được triển khai. Bộ lọc này dựa trên bộ lọc thông thấp bậc nhất (first-order low-pass filter) với tần số cắt f_c thay đổi động theo vận tốc di chuyển của tín hiệu. Cơ chế hoạt động cho một trục tọa độ x tại thời điểm t_k với khoảng thời gian $dt = t_k - t_{k-1}$ được mô tả như sau:

Đầu tiên, tính đạo hàm rời rạc để ước lượng vận tốc của tín hiệu đầu vào x_k :

$$dx_k = \frac{x_k - \hat{x}_{k-1}}{dt} \quad (5)$$

Lọc thông thấp vận tốc này để thu được vận tốc mượt mà edx_k :

$$edx_k = \alpha_d \cdot dx_k + (1 - \alpha_d) \cdot edx_{k-1} \quad (6)$$

Trong đó hệ số lọc đạo hàm α_d được tính qua tần số cắt cố định $f_{c,d}$ (thường chọn bằng 1.0 Hz):

$$\alpha_d = \frac{1}{1 + \frac{\tau_d}{dt}}, \quad \tau_d = \frac{1}{2\pi f_{c,d}} \quad (7)$$

Tần số cắt thích nghi f_c của tín hiệu chính sau đó được điều chỉnh động theo độ lớn của vận tốc mượt mà:

$$f_c = f_{c,min} + \beta \cdot |edx_k| \quad (8)$$

Trong đó $f_{c,min}$ là tần số cắt tối thiểu khi đứng yên (dùng để triệt tiêu nhiễu jitter rung tay), và β là hệ số nhảy tốc độ (tăng tần số cắt khi tay vung nhanh để giảm thiểu trễ).

Cuối cùng, tính hệ số lọc thích nghi α và giá trị tọa độ đầu ra sau lọc $\hat{x}_k$:

$$\alpha = \frac{1}{1 + \frac{\tau}{dt}}, \quad \tau = \frac{1}{2\pi f_c} \quad (9)$$

$$\hat{x}_k = \alpha \cdot x_k + (1 - \alpha) \cdot \hat{x}_{k-1} \quad (10)$$

Bộ lọc này đảm bảo hai trạng thái hoạt động tối ưu: khi tay đứng yên, f_c giảm về gần $f_{c,min}$ khiến bộ lọc làm mịn mạnh mẽ, diệt sạch nhiễu; khi tay vung nhanh để thực hiện các thao tác chém hoặc ném, f_c tăng lên giúp giảm trễ tối đa, bám sát chuyển động thực tế.

Kết chương

Chương 2 đã thiết lập toàn bộ cơ sở lý thuyết toán học và giải thuật cho hệ thống, từ OpenCV tiền xử lý ảnh, MediaPipe Hands trích xuất landmarks, mô hình mạng MLP phân loại cử chỉ chuẩn hóa, truyền thông UDP, đến bộ lọc thích nghi chống rung One Euro Filter. Dựa trên các nền tảng lý thuyết vững chắc này, Chương 3 tiếp theo sẽ trình bày chi tiết kiến trúc thiết kế hệ thống thực tế và các kết quả thực nghiệm đạt được.